

FUNCTIONAL SPECIFICATION DOCUMENT

JKN RiskGraph

Detailed Functional Requirements, User Workflows, and AI Integration

Graph Analytics-Based Risk Intelligence Platform

Version 1.0

September 2026

Healthkathon 2026

Document Control

Document Title	Functional Specification Document - JKN RiskGraph
Version	1.0
Date	September 1, 2026
Status	Draft
Related Documents	User Requirements Specification v1.0

Executive Summary

This Functional Specification Document provides detailed technical and operational specifications for the JKN RiskGraph platform. While the User Requirements Specification (URS) defines WHAT the system must do, this document explains HOW it will work, including detailed user workflows, AI integration architecture, and specific functional requirements for each system capability.

The document is organized around the six user roles defined in the URS, detailing the specific workflows, screens, interactions, and AI-assisted capabilities each role will experience. Special attention is given to explaining how artificial intelligence and machine learning are integrated throughout the platform to automate anomaly detection, risk scoring, and investigator assistance.

Table of Contents

1. Introduction
2. System Architecture Overview
 - 2.1 High-Level Architecture
 - 2.2 Data Flow Architecture
 - 2.3 AI/ML Integration Points
3. Detailed Functional Requirements by Module
 - 3.1 Data Ingestion and Processing Module
 - 3.2 Graph Construction Module
 - 3.3 Risk Detection Engine
 - 3.4 Investigation Workflow Module
 - 3.5 Reporting and Visualization Module
4. User Role Workflows and Scenarios
 - 4.1 Risk Investigator/Auditor Workflows
 - 4.2 Risk Analyst Workflows
 - 4.3 Data Manager Workflows
 - 4.4 System Administrator Workflows
 - 4.5 Supervisor/Investigation Manager Workflows
 - 4.6 Executive/Director Workflows
5. AI and Machine Learning Integration
 - 5.1 Anomaly Detection Framework
 - 5.2 Risk Scoring Algorithm
 - 5.3 Explainable AI Implementation
 - 5.4 Feedback Loop and Model Training
6. System Flow Diagrams
 - 6.1 End-to-End Investigation Flow

6.2 Data Processing Flow

6.3 Risk Detection Flow

7. Interface Specifications

7.1 Investigation Dashboard

7.2 Network Visualization Interface

7.3 Risk Explanation Panel

7.4 Analytics Workbench

8. Integration Points and APIs

9. Appendices

1. Introduction

This Functional Specification Document (FSD) serves as the detailed technical companion to the User Requirements Specification for JKN RiskGraph. While the URS establishes what the system must accomplish from a user perspective, this FSD details how those requirements translate into specific system behaviors, workflows, and technical implementations.

The document is intended for the development team, system architects, quality assurance personnel, and technical stakeholders who need to understand the detailed operational mechanics of the platform. Each section provides sufficient detail to guide implementation decisions while maintaining flexibility for technical optimization.

2. System Architecture Overview

2.1 High-Level Architecture

JKN RiskGraph follows a layered architecture pattern with clear separation of concerns across data, analytics, application, and presentation layers. The system is designed to process large-scale healthcare data, construct and analyze complex network graphs, and present actionable intelligence to users with varying technical capabilities.

Architecture Layers:

Data Layer: Handles ingestion, validation, entity resolution, and storage of claims, participant, provider, doctor, and reference data. Includes both transactional databases for operational data and a graph database for network analytics.

Analytics Layer: Contains the risk detection engine comprising rule engines, statistical anomaly detection, graph algorithms, machine learning models, and risk scoring logic. This layer processes the healthcare graph to identify suspicious patterns.

Application Layer: Implements business logic for investigation workflows, case management, user management, reporting, and integration with external systems. Orchestrates interactions between the data and analytics layers.

Presentation Layer: Provides web-based user interfaces tailored to each user role, including investigation dashboards, network visualizations, analytics workbenches, and administrative consoles.

2.2 Data Flow Architecture

Data flows through the system in a pipeline architecture, with each stage transforming and enriching the data before passing it to the next stage. The primary data flow follows this sequence:

- Stage 1: Data Ingestion - Raw claims, participant, provider, and doctor data is extracted from source systems (JKN databases, APIs, file transfers) and loaded into staging tables.
- Stage 2: Data Quality and Validation - Data quality checks identify missing values, invalid formats, referential integrity issues, and outliers. Records failing critical validation are quarantined for manual review.
- Stage 3: Entity Resolution - Deduplication and entity linking processes identify and merge records representing the same real-world entity across different source systems. Probabilistic matching algorithms handle variations in names, identifiers, and attributes.
- Stage 4: Graph Construction - Clean, resolved entities and their relationships are transformed into graph structures. Nodes represent entities (participants, doctors, providers, claims) and edges represent relationships (visits, treatments, referrals, claim submissions).

- Stage 5: Feature Engineering - Graph-based features are computed for each entity and relationship, including node degrees, centrality measures, community membership, temporal patterns, and peer group statistics.
- Stage 6: Risk Detection - The risk detection engine applies rules, statistical tests, graph algorithms, and machine learning models to identify anomalous patterns. Multiple detection methods run in parallel and their outputs are fused into unified risk scores.
- Stage 7: Risk Prioritization - Detected risks are ranked by score, impact, confidence, and investigator feedback history. High-priority networks and entities are surfaced in investigation queues.
- Stage 8: Explainability Generation - For each flagged network or entity, the system generates explanations detailing which detection methods fired, what specific patterns triggered alerts, and how peer comparison informed the risk assessment.
- Stage 9: Investigation and Feedback - Investigators review prioritized risks, drill into evidence, make decisions (confirm, dismiss, escalate), and provide feedback. Feedback is captured and flows back into the risk detection engine for model improvement.

2.3 AI/ML Integration Points

Artificial intelligence and machine learning are integrated at multiple points throughout the system architecture, not as a monolithic "AI module" but as targeted capabilities addressing specific analytical challenges. The key integration points are:

Entity Resolution (Supervised ML): Machine learning models predict whether two entity records refer to the same real-world entity based on similarity of attributes. Models are trained on manually labeled duplicate/non-duplicate pairs and applied to fuzzy-match candidates during data processing.

Statistical Anomaly Detection (Unsupervised ML): Isolation Forest and Local Outlier Factor algorithms identify entities whose behavior deviates significantly from peer groups without requiring labeled training data. These unsupervised methods detect novel patterns not anticipated in rules.

Graph Community Detection (Graph Algorithms): Louvain and Label Propagation algorithms partition the healthcare graph into communities of closely connected entities. Anomalous communities (unusual density, composition, or inter-community connections) are flagged for review.

Risk Scoring (Ensemble ML): Gradient boosting models (LightGBM, XGBoost) combine graph features, statistical features, and rule outputs to predict entity-level and network-level risk. Models are trained on historical investigation outcomes (confirmed vs dismissed cases).

Graph Neural Networks - Future (Deep Learning): Graph Neural Networks (GNNs) like GraphSAGE and Graph Attention Networks can learn node embeddings that capture complex network patterns. While not in MVP scope, GNN models will be evaluated for improved detection accuracy based on research findings like Muhammad et al. (2025).

Natural Language Generation for Explanations (Rule-Based + Templates): Risk explanations are generated using template-based natural language generation that translates technical risk signals into human-readable narratives. Templates are populated with specific values (percentages, peer comparisons, flagged relationships).

Investigation Assistant (LLM Integration - Future): Large language models can be integrated to provide conversational investigation assistance, answering questions about networks, synthesizing evidence from multiple sources, and suggesting investigation strategies. This capability is planned for future phases.

3. Detailed Functional Requirements by Module

3.1 Data Ingestion and Processing Module

Functional Requirement ID: FR-DATA-001 to FR-DATA-020

The Data Ingestion and Processing Module is responsible for extracting data from JKN source systems, validating data quality, resolving entity identities, and preparing clean datasets for graph construction. This module operates on a scheduled batch basis (daily or weekly depending on data refresh requirements).

Detailed Requirements:

FR-DATA-001: Claim Data Ingestion: The system shall ingest claim records including claim_id (unique identifier), participant_id, provider_id, doctor_id, claim_date, claim_amount, diagnosis_codes (array), procedure_codes (array), length_of_stay, admission_date, discharge_date, and claim_status. Data source is JKN claims database via database connection or API. Ingestion frequency is daily incremental loads with full refresh monthly.

FR-DATA-002: Participant Data Ingestion: The system shall ingest participant master data including participant_id (unique identifier), name_hash (hashed for privacy), date_of_birth, gender, region_code, employer_id, registration_date, and participant_status. Personally identifiable information (PII) such as names must be hashed before storage. Data source is JKN participant registry.

FR-DATA-003: Provider Data Ingestion: The system shall ingest provider master data including provider_id (unique identifier), provider_name, facility_type (hospital, clinic, FKTP, etc.), region_code, ownership_type, bed_count, services_offered (array), and accreditation_level. Data source is JKN provider registry.

FR-DATA-004: Doctor Data Ingestion: The system shall ingest doctor master data including doctor_id (unique identifier), doctor_name, specialty, license_number, affiliated_providers (array of provider_ids), and practice_start_date. Data source is JKN doctor registry.

FR-DATA-005: Data Validation - Completeness: The system shall validate that required fields are present and non-null for each record type. Claims missing participant_id, provider_id, claim_date, or claim_amount shall be flagged as critical errors and quarantined. Other missing fields shall be flagged as warnings but not block processing.

FR-DATA-006: Data Validation - Format: The system shall validate that field values conform to expected data types and formats. Date fields must parse as valid dates, numeric fields must be numeric, ID fields must match expected patterns, and code fields must reference valid code tables.

FR-DATA-007: Data Validation - Referential Integrity: The system shall validate that foreign key references are valid. Claims must reference valid participants, providers, and doctors. Orphaned records (referencing non-existent entities) shall be flagged and either auto-corrected (if entity can be inferred) or quarantined.

FR-DATA-008: Data Validation - Business Rules: The system shall validate business rules including: `claim_amount > 0`, `length_of_stay >= 0`, `discharge_date >= admission_date`, `claim_date` within reasonable window of service date, and participant age at service date is consistent with `date_of_birth`.

FR-DATA-009: Entity Resolution - Participant Deduplication: The system shall identify and merge duplicate participant records that likely represent the same individual. Matching criteria include exact `participant_id` match (trivial case), fuzzy name match + `date_of_birth` match (high confidence), and probabilistic matching on name + gender + region (medium confidence). Machine learning models shall score candidate matches and auto-merge above confidence threshold.

FR-DATA-010: Entity Resolution - Provider Deduplication: The system shall identify and merge duplicate provider records. Matching criteria include exact `provider_id` match, facility name fuzzy match + region match, and manual review for ambiguous cases. A master provider ID shall be assigned to each cluster of duplicates.

FR-DATA-011: Entity Resolution - Doctor Deduplication: The system shall identify and merge duplicate doctor records. Matching criteria include exact `doctor_id` match, name fuzzy match + specialty match + affiliated provider match, and license number match (when available). A master doctor ID shall be assigned to each cluster.

FR-DATA-012: Data Quality Dashboard: The system shall provide a data quality dashboard accessible to Data Managers showing: total records ingested by type and date,

validation error counts by type and severity, entity resolution statistics (duplicates found, auto-merged, manual review pending), data completeness metrics by field, and data freshness (time since last successful load).

FR-DATA-013: Quarantine Management: The system shall maintain a quarantine area for records failing critical validation. Data Managers can review quarantined records, manually correct data issues, approve or reject records, and track quarantine resolution metrics.

FR-DATA-014: Data Lineage Tracking: The system shall track data lineage for each entity and relationship in the graph, recording source system, load timestamp, entity resolution decisions applied, and any manual corrections. Lineage metadata shall be queryable for audit purposes.

FR-DATA-015: Incremental Load Processing: The system shall support incremental data loads that process only new or changed records since the last load. Change detection shall be based on source system timestamps or change data capture (CDC) mechanisms.

FR-DATA-016: Full Refresh Processing: The system shall support periodic full refresh loads that rebuild the entire graph from scratch. Full refresh shall include: truncate existing graph data, reload all historical data within configured window (e.g., 12 months), re-run entity resolution across all data, and re-compute all graph features and risk scores.

FR-DATA-017: Data Anonymization: The system shall anonymize or pseudonymize personally identifiable information in accordance with data privacy regulations. Name fields shall be hashed using a salted hash function, date_of_birth shall be stored as age or age band, and region shall be generalized to broader geographic areas if needed.

FR-DATA-018: Audit Logging: The system shall log all data ingestion and processing activities including: load start and end times, record counts by type, validation error counts, entity resolution decisions, and user actions (manual corrections, approvals). Logs shall be retained for audit and compliance purposes.

FR-DATA-019: Error Notification: The system shall notify Data Managers when data loads fail, validation error rates exceed thresholds, or quarantine queues require attention. Notifications shall be sent via email or system alerts with sufficient detail to triage issues.

FR-DATA-020: Performance Optimization: The system shall process data ingestion and entity resolution within acceptable timeframes: daily incremental loads shall complete within 4 hours, full monthly refresh shall complete within 24 hours. Processing shall be parallelized and optimized for large-scale data volumes.

3.2 Graph Construction Module

Functional Requirement ID: FR-GRAPH-001 to FR-GRAPH-015

The Graph Construction Module transforms clean, resolved entity data into a heterogeneous graph structure where nodes represent entities (participants, doctors, providers, claims) and edges represent relationships and interactions (visits, treatments, referrals, claim submissions). The graph database serves as the foundation for all network analytics and risk detection.

FR-GRAPH-001: Node Creation - Participants: The system shall create a participant node for each unique participant in the data. Node properties shall include: `participant_id` (unique identifier), `age_band` (to protect privacy), `gender`, `region`, `employer_type`, and `registration_year`. `Name` and `date_of_birth` shall NOT be stored as node properties to protect PII.

FR-GRAPH-002: Node Creation - Doctors: The system shall create a doctor node for each unique doctor. Node properties shall include: `doctor_id`, `specialty`, `years_in_practice` (calculated from `practice_start_date`), `affiliated_provider_count` (number of providers where doctor practices), and `total_patient_count`.

FR-GRAPH-003: Node Creation - Providers: The system shall create a provider node for each unique provider. Node properties shall include: `provider_id`, `provider_name`, `facility_type`, `region`, `ownership_type`, `bed_count` (for hospitals), `accreditation_level`, and `service_categories` (array of service types offered).

FR-GRAPH-004: Node Creation - Claims: The system shall create a claim node for each claim record. Node properties shall include: `claim_id`, `claim_date`, `claim_amount`, `diagnosis_codes`, `procedure_codes`, `length_of_stay`, and `claim_status`. Claims are connected to their associated participants, doctors, and providers via edges.

FR-GRAPH-005: Edge Creation - VISITS Relationship: The system shall create a VISITS edge from each participant node to each provider node when the participant has submitted at least one claim at that provider. Edge properties shall include: `first_visit_date`, `last_visit_date`, `total_visits` (count of distinct claim dates), and `total_amount` (sum of all claim amounts).

FR-GRAPH-006: Edge Creation - TREATS Relationship: The system shall create a TREATS edge from each doctor node to each participant node when the doctor has treated that participant. Edge properties shall include: first_treatment_date, last_treatment_date, treatment_count, and diagnosis_codes_treated (array of unique diagnosis codes in treatments).

FR-GRAPH-007: Edge Creation - WORKS_AT Relationship: The system shall create a WORKS_AT edge from each doctor node to each provider node where the doctor is affiliated. Edge properties shall include: affiliation_start_date, affiliation_end_date (null if active), patient_count_at_provider (number of unique patients treated by this doctor at this provider), and claim_count_at_provider.

FR-GRAPH-008: Edge Creation - GENERATES Relationship: The system shall create a GENERATES edge from each participant node to each claim node. This relationship is one-to-one (each claim is generated by exactly one participant). Edge properties shall include: claim_date and is_flagged_by_participant (boolean for participant-level risk signals).

FR-GRAPH-009: Edge Creation - SUBMITS Relationship: The system shall create a SUBMITS edge from each provider node to each claim node submitted by that provider. Edge properties shall include: submission_date and is_flagged_by_provider (boolean for provider-level risk signals).

FR-GRAPH-010: Graph Database Selection: The system shall use a graph database capable of handling millions of nodes and tens of millions of edges with acceptable query performance. Neo4j (commercial or community edition) is recommended for MVP. Alternative options include Memgraph or PostgreSQL with graph extensions (Apache AGE).

FR-GRAPH-011: Graph Refresh Strategy: The system shall support incremental graph updates where new data is added as new nodes and edges without rebuilding the entire graph. For data corrections or deletions, the system shall mark nodes/edges as soft-deleted rather than physically removing them (to preserve audit trail).

FR-GRAPH-012: Graph Query Performance: The system shall index frequently-queried node properties (participant_id, doctor_id, provider_id, claim_date) and edge types to

ensure acceptable query performance. Complex graph queries (e.g., 3-hop network expansion) shall complete within 10 seconds for typical use cases.

FR-GRAPH-013: Graph Backup and Recovery: The system shall backup the graph database daily. Backups shall be tested periodically to ensure recoverability. Point-in-time recovery shall be supported for rolling back to previous graph states if needed.

FR-GRAPH-014: Graph Versioning: The system shall maintain version metadata for the graph indicating when it was last built/updated, what data time window it covers, and what data quality validations were applied. This allows users to understand what data they are analyzing.

FR-GRAPH-015: Graph Export and Sharing: The system shall provide capabilities for Risk Analysts to export subgraphs (e.g., a specific suspicious network) for external analysis or sharing. Export formats shall include GraphML, JSON, or CSV edge lists.

3.3 Risk Detection Engine

Functional Requirement ID: FR-RISK-001 to FR-RISK-030

The Risk Detection Engine is the core intelligence component of JKN RiskGraph. It applies multiple detection methods in parallel - domain-specific rules, statistical tests, graph algorithms, and machine learning models - to identify anomalous patterns in the healthcare graph. The engine produces risk signals, scores, and explanations that are surfaced to investigators.

This module is divided into several sub-components:

A. Rule Engine (Domain Knowledge)

FR-RISK-001: Rule Definition Framework: The system shall provide a rule definition framework allowing Risk Analysts to define detection rules in a structured format. Rules shall specify conditions (e.g., IF participant has >5 claims with identical diagnosis and procedure within 30 days THEN flag as potential cloning) and output signals (risk type, confidence level, contributing entities).

FR-RISK-002: Cloning Detection Rule: The system shall implement a cloning detection rule that identifies sets of claims with highly similar structures. Specifically: identify claims within the same provider and time window (30 days), compare claim attributes (diagnosis codes, procedure codes, claim amount, length of stay), flag as potential cloning if similarity score >90% and involves different participants. Output signal: CLONING_PATTERN with list of similar claim IDs.

FR-RISK-003: Repeat Billing Detection Rule: The system shall implement a repeat billing detection rule that identifies duplicate billing for the same service. Specifically: identify claims with same participant, same provider, same procedure code within care episode window (7 days for outpatient, 30 days for chronic care), flag if claims are not marked as legitimate follow-up visits. Output signal: REPEAT_BILLING with claim pair IDs and time gap.

FR-RISK-004: Referral Concentration Detection Rule: The system shall implement a referral concentration detection rule that identifies doctors or FKTPs referring unusually high percentages of patients to specific providers. Specifically: for each doctor/FKTP, compute percentage of referrals to each provider, compare to peer group median (doctors with same specialty and region), flag if concentration >2 standard deviations above peer median. Output signal: REFERRAL_CONCENTRATION with doctor, target provider, percentage, and peer comparison.

FR-RISK-005: Prolonged Length of Stay Detection Rule: The system shall implement a prolonged LOS detection rule that identifies providers with unusually long average length of stay. Specifically: for each provider and diagnosis code, compute average LOS, compare to peer group median (providers with same facility type and region), flag if average LOS >2 standard deviations above peer median. Output signal: PROLONGED_LOS with provider, diagnosis, average LOS, and peer comparison.

FR-RISK-006: Rule Execution Engine: The system shall execute all active rules against the graph on a scheduled basis (after each graph update). Rule execution shall be parallelized to minimize processing time. Rule outputs (signals) shall be stored with timestamps and linked to the specific entities/relationships that triggered the rule.

B. Statistical Anomaly Detection

FR-RISK-007: Peer Group Definition: The system shall define peer groups for providers based on facility_type, region, service_mix, and patient_volume. Providers within each peer group are compared against each other to detect outliers. Peer group assignments shall be reviewed and adjusted by Risk Analysts.

FR-RISK-008: Feature Computation for Anomaly Detection: The system shall compute statistical features for each entity including: provider claim_rate (claims per month), average_claim_amount, procedure_distribution (percentage of each procedure code), diagnosis_distribution, patient_turnover_rate, and length_of_stay statistics. Features shall be computed over rolling time windows (30 days, 90 days, 12 months).

FR-RISK-009: Univariate Outlier Detection: The system shall apply univariate outlier detection methods (Z-score, IQR method) to identify providers whose behavior deviates significantly from their peer group on individual metrics. Specifically: for each feature and peer group, compute mean and standard deviation, flag entities >3 standard deviations from mean as outliers. Output signal: STATISTICAL_OUTLIER with feature name, entity value, peer mean, and deviation.

FR-RISK-010: Multivariate Anomaly Detection (Isolation Forest): The system shall apply Isolation Forest algorithm to detect providers whose multivariate feature vectors are anomalous relative to their peer group. Isolation Forest is an unsupervised algorithm that identifies instances requiring fewer splits to isolate. Model shall be trained on peer group

data and applied to score each provider. Anomaly scores >0.7 shall trigger MULTIVARIATE_ANOMALY signal.

FR-RISK-011: Temporal Pattern Anomaly Detection: The system shall detect anomalous temporal patterns including sudden spikes or drops in activity, unusual day-of-week patterns, and seasonal anomalies. Time series analysis methods (moving averages, change point detection) shall identify when entity behavior deviates from historical baseline.

C. Graph Analytics

FR-RISK-012: Degree Centrality Computation: The system shall compute degree centrality for all nodes, measuring how many connections each entity has. High-degree nodes (e.g., providers with unusually many patient connections, doctors with excessive referral partners) shall be flagged for review. Output signal: HIGH_DEGREE_CENTRALITY with node, degree, and peer comparison.

FR-RISK-013: Betweenness Centrality Computation: The system shall compute betweenness centrality for all nodes, measuring how often a node lies on the shortest path between other nodes. High-betweenness nodes often act as bridges or gatekeepers in networks and may indicate coordination roles in fraud schemes. Output signal: HIGH_BETWEENNESS_CENTRALITY with node and score.

FR-RISK-014: Community Detection (Louvain Algorithm): The system shall apply community detection algorithms (Louvain method recommended) to partition the graph into densely connected communities. Communities shall be analyzed for anomalous characteristics including: unusually high internal claim density, concentration of specific diagnosis/procedure codes, and high proportion of high-value claims. Output signal: ANOMALOUS_COMMUNITY with community ID, member count, and anomaly description.

FR-RISK-015: Motif Detection: The system shall detect recurring subgraph patterns (motifs) that may indicate coordinated fraud. Specific motifs to detect include: Referral triangle (Doctor A refers to Provider B who refers to Pharmacy C), Claim cloning star (one participant generates multiple identical claims), and Provider cluster (multiple providers with overlapping patient sets and similar claim patterns). Output signal: SUSPICIOUS_MOTIF with motif type, involved entities, and pattern description.

FR-RISK-016: Path Analysis: The system shall analyze paths through the graph to identify suspicious referral chains, circular referral patterns (Provider A → Provider B → Provider A), and unusually long referral chains that may indicate unnecessary intermediaries. Output signal: SUSPICIOUS_PATH with path description and entities involved.

D. Machine Learning Models

FR-RISK-017: Feature Engineering for ML: The system shall engineer features for machine learning models combining graph features (degrees, centrality, community membership), statistical features (claim rates, amounts, distributions), temporal features (activity patterns, growth rates), and rule outputs (count of rules triggered, types of signals). Feature vectors shall be computed for each entity to be scored.

FR-RISK-018: Training Data Preparation: The system shall prepare training data from historical investigation outcomes. Each investigated entity shall be labeled as CONFIRMED_RISK (investigation found issues) or DISMISSED (investigation found no issues). Training data shall include feature vectors and labels, split into training (70%), validation (15%), and test (15%) sets.

FR-RISK-019: Risk Scoring Model Training (Gradient Boosting): The system shall train gradient boosting models (LightGBM or XGBoost) to predict risk probability for each entity. Models shall be trained separately for provider risk, doctor risk, and network risk. Hyperparameters shall be tuned via cross-validation. Model performance metrics (AUC-ROC, precision, recall, F-score) shall be tracked.

FR-RISK-020: Model Deployment and Scoring: The system shall deploy trained models to score all entities in the graph after each graph update. Model predictions (risk probabilities) shall be stored alongside other risk signals. Models shall be versioned and older versions retained for reproducibility and rollback.

FR-RISK-021: Model Retraining Pipeline: The system shall support periodic model retraining incorporating new investigation feedback. Retraining shall occur monthly or when sufficient new labeled data is available (minimum 100 new labels). Model performance before and after retraining shall be compared to ensure improvement.

FR-RISK-022: Graph Neural Network (GNN) Experimentation - Future: The system architecture shall accommodate future integration of Graph Neural Network models (e.g., GraphSAGE, GAT, HINormer) for advanced pattern detection. GNN models can learn node embeddings that capture complex network structure and relationships. Based on research (Muhammad et al., 2025), GNN architectures have achieved 82-84% F-scores on healthcare fraud detection. GNN integration is scoped for post-MVP evaluation.

E. Risk Fusion and Prioritization

FR-RISK-023: Risk Signal Fusion: The system shall fuse multiple risk signals (from rules, statistical methods, graph algorithms, ML models) into unified risk scores. Fusion logic shall: assign weights to each signal type based on confidence and historical accuracy, aggregate weighted signals for each entity, normalize to 0-100 scale. Fusion weights shall be configurable by Risk Analysts.

FR-RISK-024: Risk Score Calibration: The system shall calibrate risk scores to ensure they are interpretable and actionable. Calibration shall map score ranges to risk categories: 0-29 LOW, 30-59 MEDIUM, 60-79 HIGH, 80-100 CRITICAL. Score distribution shall be monitored to avoid score inflation or deflation over time.

FR-RISK-025: Network-Level Risk Aggregation: The system shall aggregate entity-level risk scores to compute network-level risk scores. For a connected subgraph (network): identify all entities within the network, compute weighted average of entity risk scores (weighted by entity centrality or claim volume), apply network-specific risk factors (size, density, temporal synchronization), output network risk score. Networks with high scores represent coordinated schemes.

FR-RISK-026: Risk Prioritization Algorithm: The system shall prioritize detected risks for investigation using a multi-criteria scoring function considering: risk score magnitude, financial impact (total claim amount involved), novelty (new pattern not seen before), and investigator feedback history (avoid repeatedly surfacing previously dismissed patterns). Top-N highest priority items populate investigation queues.

FR-RISK-027: Risk Threshold Configuration: The system shall allow Risk Analysts to configure risk score thresholds for different actions: auto-queue for investigation, escalate to supervisor, generate alert notification. Thresholds shall be adjustable per risk type to account for different false positive rates.

FR-RISK-028: False Positive Tracking: The system shall track false positive rates (risks flagged but dismissed by investigators) by risk type and signal source. High false positive rates shall trigger review of detection rules/models for refinement.

FR-RISK-029: Risk Detection Performance Monitoring: The system shall monitor risk detection performance metrics including: total risks detected per period, risk distribution by type and severity, investigation conversion rate (% of flagged risks confirmed as actual issues), model accuracy on holdout test set, processing time for risk scoring run. Metrics shall be accessible to Risk Analysts and System Administrators.

FR-RISK-030: Explainability Generation: The system shall generate human-readable explanations for each risk detection detailing: what specific patterns triggered the risk, which detection methods (rules, models, algorithms) contributed to the score, how the entity compares to peers (percentiles, deviations), and which related entities are involved. Explanations shall be displayed alongside risk scores in investigator interfaces.